Shorts Studio

The complete guide

You do not need to know how to code to use this. If you can copy and paste, you can run it.

Generated 2026-10-12

This guide covers everything: what the tool is, how to start it, how to make a video, how to keep your work safe with Git, and how to run the same tool on a second computer.

Contents

1. What this tool actually is
2. Setting up, once
3. Starting it, every time
4. Making a video — the seven steps
    - Diagrams that are worked out, not guessed
    - Pictures that are checked, and the test bench
    - Instagram and Facebook
    - Making a thumbnail
    - A carousel post
    - The upload kit
    - Using Claude instead of Gemini
    - Writing the script yourself
    - Real artwork in every layout
    - Animation comes from your verbs
    - Scenes that move
    - Moving backdrops
    - Drawing your own backdrops
    - The Doodle look and the mascot
    - Cuts, text and the finishing layer
    - Looks from Claude Design
    - Aptitude and reasoning videos
    - A note on units
5. Git — your undo button
6. GitHub — your backup and your bridge
7. Running it on another computer
8. What it costs
9. When something goes wrong
    - When a port stays stuck
    - Checking the build itself
10. Where things are saved
11. Questions people ask
12. A checklist for good videos

---

1. What this tool actually is

You pick a subject, press a button, and about three minutes later you have a finished video with a real human-sounding voiceover, ready to upload. Or you write the script yourself and let the tool do the rest — see Writing the script yourself.

Four services do the work, and they all run from one page in your browser:

	What it does	Needed?
Google Gemini	Writes the question, the four options, the explanation and the exact words the narrator says — or, in explainer mode, the whole storyboard. At the end it writes your title, tags and description for **YouTube, Instagram and Facebook**, and can **design the thumbnail**. With billing enabled it also **draws pictures** — scene backdrops and the thumbnail art.	Required
Anthropic Claude	An alternative writer for the question or the storyboard. Pick which one on step 2.	Optional
ElevenLabs	Turns that script into speech. Can also draw backdrops, but only on a Pro plan.	Required
DeepSeek	Solves the question independently and says whether it agrees with Gemini.	Optional
Pexels + NASA	Free photos to sit behind the text. NASA needs no key.	Optional

The video itself is drawn on your own computer by **Remotion**, which is why rendering costs nothing.

Two ways to tell it

- **Quiz** — a question, four options, a countdown and the reveal. The format that stops a scroll.
- **Explainer** — no question at all. A storyboard that builds understanding scene by scene using analogies and diagrams: a title card, an analogy, a labelled diagram, the steps, a comparison, a timeline, a scene where things actually move, and a recap. Aimed at 3 to 5 minutes in 16:9.

Three kinds of video

- **Curiosity STEM** — counter-intuitive science and maths for a general audience.
- **Electrical exam prep** — in the style of a real paper, aimed at a specific exam: GATE EE, ESE/IES, SSC JE, RRB JE, State AE/JE, PSU (UPPCL/DMRC/NTPC/BHEL), or ITI/Wireman.
- **Aptitude & reasoning** — the quant, reasoning, English and awareness sections that nearly every competitive paper carries, aimed at SSC, banking, RRB, CAT, campus placements, GATE GA, CSAT, defence, state PSC or the teaching exams. See Aptitude and reasoning videos.

Two shapes

- **Portrait 9:16** — Shorts, Reels, TikTok. 30 to 90 seconds.
- **Landscape 16:9** — a proper explainer for YouTube. 2 to 5 minutes.

Alongside either, step 7 can make a **square carousel post** of the same question — 1080 × 1080 slides for Instagram and Facebook. See A carousel post.

How long does one video take? Roughly 3–4 minutes of your attention, most of it waiting. The very first video takes longer, because the tool downloads its rendering engine once.

2. Setting up, once

Node.js

This is the program that runs everything. Check whether you already have it — press the **Windows key**, type `powershell`, press **Enter**, then type:

```
node -v
```

A reply like `v20.11.0` or higher means you are fine. An error means you need it: go to nodejs.org, download the big green **LTS** button, click Next through the installer, then **close and reopen PowerShell**.

Your API keys

An "API key" is a long password that lets this tool use an online service on your behalf.

Gemini (required — writes the questions)

1. Go to https://aistudio.google.com/app/apikey
2. Sign in with any Google account → **Create API key** → copy it.
3. It looks like `AIzaSyD…`

ElevenLabs (required — does the voice)

1. Go to https://elevenlabs.io and make a free account.
2. Go to https://elevenlabs.io/app/settings/api-keys → **Create API key** → copy it.
3. It looks like `sk_1a2b3c…`

Claude (optional — an alternative writer)

1. Go to https://console.anthropic.com → **API Keys** → **Create Key** → copy it.
2. It looks like `sk-ant-…`

Only needed if you want Claude to write the questions or storyboards instead of Gemini. See Using Claude instead of Gemini.

DeepSeek (optional — double-checks the answer)

1. Go to https://platform.deepseek.com/api_keys → **Create new API key** → copy it.
2. It looks like `sk-1a2b3c…`

Worth adding. Gemini writes the question *and* marks its own answer, so nothing catches a confidently wrong one. A check costs a fraction of a cent.

Pexels (optional — backdrop photos)

1. Go to https://www.pexels.com/api/ → sign up free → **Get Started** → copy the key.

Without it you still get NASA's public-domain library, which is excellent for space and physics but thin for chemistry and biology.

> **Keep these private.** Anybody holding your key can spend your credits. Never put one in a screenshot, a video, or a message. They are stored only in your browser, never in a file — which is also why they are never uploaded to GitHub.

Install the packages

Open PowerShell and go to the project:

```
cd C:\Projects\shorts-studio
```

Then:

```
npm install
```

This takes one to three minutes. Yellow warnings are normal — only a red `ERR!` means trouble.

If you see "running scripts is disabled on this system"

> This is a Windows security default, not a fault in the tool. Two ways past it.
>
> **The easy way, changing nothing** — add `.cmd`:
>
> `npm.cmd install`
> Use `npm.cmd` everywhere this guide says `npm`. That is the whole fix.
>
> **The permanent way** — run this once and press `Y`:
>
> `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
> `RemoteSigned` lets scripts written on your own computer run, while anything downloaded from the internet must still be signed. It applies to your account only, and it is what Microsoft recommends for people who develop on Windows.

---

3. Starting it, every time

```
cd C:\Projects\shorts-studio
```

```
npm start
```

Two things start together: the **helper**, which talks to Gemini and ElevenLabs and does the rendering, and the **web page** you actually use. The helper opens your browser once it is ready, so give it a few seconds. If it does not appear, paste this in yourself:

```
http://localhost:5173/
```

That address never changes. If something else is already using it the tool stops and says so, rather than quietly moving to another one.

> **"Starting up..." for a second or two is normal.** The page loads faster than the helper does, so on a cold start it waits for it and tells you. It clears by itself. Only if it is still trying after twenty seconds will it tell you the helper is genuinely missing.

Leave the PowerShell window open the whole time. That black window *is* the engine. Closing it stops the tool. When you are finished for the day, click it and press `Ctrl + C`.

---

4. Making a video — the seven steps

Seven numbered buttons run across the top. You move left to right. Steps you have not earned yet stay greyed out on purpose, so you cannot get lost.

Step 1 — Keys

Paste your keys and press **Test my keys**. You want green ticks. It also reports how many ElevenLabs voice credits you have left this month.

You do this once, ever. If a key is rejected, the cause is almost always a stray space — clear the box and paste again.

Step 2 — Topic

Where you decide what the video is about.

Setting	What it does
How it is told	**Quiz** or **Explainer**. Choosing Explainer switches to 16:9 and at least three minutes, because that is what the format needs.
What kind of video	**Curiosity STEM** for a general audience, **Electrical exam prep** for candidates revising, or **Aptitude & reasoning** for the quant, reasoning, English and awareness sections.
Exam *(both exam modes)*	Sets the depth. GATE and ESE want derivation and analysis; SSC and RRB want standard formulas at speed; ITI wants practical wiring and safety with no calculus. In aptitude mode the list swaps to the aptitude papers, where CAT hides an insight and SSC stays quick and clean.
Format	Portrait or landscape. **Pick this first** — it changes how much script is written and how the frame is laid out.
Subject	The broad area. In exam mode this is the syllabus section.
Sub-topic	A dropdown of suggestions for that subject — 38 for Power Generation, for example. Pick one to concentrate a run of videos on a single section, or leave it on *Any*.
Or type your own topic	Free text. Whatever is in this box is what gets used; clear it to let Gemini choose.
Who is watching	Sets vocabulary and assumed background.
Difficulty	*Very easy* through *Brutal*.
Question style	*Mathematical* = the viewer must calculate. *Theoretical* = they must reason. *Real-world* = anchored in daily life.

Setting	What it does
Narration language	The language spoken *and* shown. Kannada, Hindi, Tamil and Telugu render in their own script — pair with a matching voice in step 4 and the Multilingual v2 model.
Curiosity factor	The most important dial. **8 or 9** is the sweet spot: counter-intuitive enough to make people comment.
Target length	40–50s performs best for Shorts. 180–240s is the sweet spot for explainers.
How many diagrams	*Balanced* (the default) gives a scene a chart, circuit or animation only when it shows exactly what that scene says, which is about half of them. *Rich* asks for one wherever there is something specific to show. *Sparse* keeps one or two. No setting asks for a diagram on every scene any more: asked for a picture it does not have, the model draws the nearest one it does. A scene with none is normal.
Gemini model	Flash is fast and cheap. Pro is worth it for hard maths. The list is built from your own key, so it only offers models you can actually use.

Curiosity high — what is trending now

Press **What is trending now?** and Gemini *searches the live web* — it is not answering from memory — then judges which of what it finds would actually make a good video. You get eight ideas, each with why it is being talked about and the counter-intuitive angle to build around. Click one and it fills the topic box.

It searches your domain: science and technology in curiosity mode, grid, generation, storage and standards news in electrical mode, or exam calendars, paper-pattern changes and the current affairs a general awareness section actually asks about in aptitude mode. **Where this came from** lists the pages it actually read, so you can check a claim yourself.

> **Trending is not the same as true.** A story spreading fast is exactly the kind that turns out to be half right. Verify the answer in step 3, and run the DeepSeek check if you have a key.

This needs a **2.5 model** (Flash or Pro) — older models cannot search. It costs one Gemini request.

Your intro (optional). Type a greeting — "*Hi, it's Hemanth here. Ready for today's question?*" — and it becomes the first thing the video says. There are one-click presets. **Gemini never rewrites this**: whatever you type is spoken and shown word for word.

Press **Generate the question**. Five to twenty seconds.

Telling it exactly what you want

The dropdowns cover the common choices. **Your own instructions** covers everything else — a box with room to write, at the bottom of step 2.

```
Use an Indian everyday example wherever possible.
Name the formula before using it, and show the unit at every step.
Make one wrong option the mistake of forgetting the root three.
```

Where it lands matters, and it is placed for you. Your instructions go in near the end of the request — after every line of built-in guidance they might need to overrule, and before the target length. So "*keep the tone dry*" beats the tone you picked from the dropdown, and "*write twenty scenes*" does not beat the length you asked for. Length and the output format are enforced outside the prompt entirely, so nothing you write here can break a

generation.

Buried in the middle, as it used to be, a line here was one bullet against eighty-odd lines of fixed direction, and quietly lost to it.

See exactly what will be sent. Open that panel under the box and you get the real request, built by the same code the Generate button uses — not an impression of it, and not a second version that can drift. It updates as you change the form.

It shows two things: **your request**, assembled from the settings, and **the standing instructions** it gets added to. The second is the one worth reading once. Most "why did it do that?" moments are a line in there you did not know existed, and once you have seen it you can write an instruction that actually overrules it.

Nothing is sent and nothing is charged by looking. It works for both kinds of video — a quiz shows its question request, an explainer its storyboard request.

Step 3 — Script

You check Gemini's homework. **This is the most important step and the one people skip.**

> **Only the spoken words appear on screen.** The narration *is* the on-screen text — shown a few words at a time, each word lighting up as it is said. There is no separate headline to keep in step, and no labels, step counters or subject chips cluttering the frame.

Every line is editable. **Read the question and satisfy yourself the marked answer is right** — AI is confidently wrong sometimes, and ten seconds here saves publishing something embarrassing.

The second opinion. With a DeepSeek key you get a **Check the answer** button:

Badge	Meaning
✓ **DeepSeek agrees**	Both models got the same answer.
⚠ **Reservations**	The answer stands, but something needs a look.
✕ **DeepSeek disagrees**	They got **different answers**. One is wrong.

On a disagreement you get a one-click button to switch the marked answer — but read the reasoning first. DeepSeek is not automatically right either; the value is knowing to look. Pick **Reasoner** for hard maths.

Edit the question afterwards and the badge is replaced by "*the check is out of date*" — a stale tick is never left pretending the new version was verified.

The diagram check. Many electrical and aptitude questions come with a diagram of the question itself — the circuit, the phasors, the price bars. Its numbers are *worked out*, not written, so the diagram doubles as a free second opinion on the answer. A card under the question says what happened; see Diagrams that are worked out, not guessed.

The picture check. Every other diagram in the script is read against the line it sits on, by a reviewer that has the frame in front of it. It runs on its own as soon as there is a script, and anything it calls wrong, or merely about the same subject, is taken out before you spend anything on the voice. Each one is listed with the reason and a **Put it back** button, because you outrank the reviewer. See Pictures that are checked, and the test bench.

> **On long explainers.** A five-minute video is a lot of script for one reply, and the room the model is given used to be left at the provider's default — 8,192 tokens, whatever the model was capable of. On a thinking model the reasoning is spent out of that first, so the longest videos were the ones most likely to stop mid-scene. It is now set explicitly, to four times what the longest storyboard needs with the whole thinking budget on top. The black window prints the tokens each generation actually used against that ceiling, so you can see how close it came.

Watch the length. A bar shows how many seconds you have written against the length you asked for. If a generated script came out well under target you also get a warning — regenerate now, while it is still free.

> The bar is an estimate at the voice's own speaking pace. **You cannot set how long a scene lasts**, and there is no slider for it anywhere. A scene lasts exactly as long as its recorded narration — that is the whole reason the words on screen stay locked to the voice. The real length arrives with the voiceover in step 4.

Rearranging the video. Every scene has controls in its header:

Control	What it does
The **kind** dropdown	What the scene is for — hook, question, options, countdown, answer, explanation, outro.
↑ ↓	Move the scene earlier or later.
×	Delete it.
+ Add a scene	Adds an explanation scene before the outro.

Changing any of these means the voiceover has to be recorded again, and the page tells you so if you have already made one. **Editing the words of a scene is cheaper:** only that scene loses its recording, and step 4 records just that one — the rest keep their voice.

The fun fact. The fact is not shown on a card of its own — the last scene *says* it, and the narration is what appears on screen. So the box and the last scene are linked: while that scene begins with the fact, typing in the box changes the scene with it, and the hint under the box says "*The last scene says this*". If the last scene says something else — Gemini sometimes words the closing line itself, or you rewrote it — the box warns you, quotes what the scene actually says, and offers **Use my fun fact there**, which puts your fact at the start of that scene. Without that, changing the box alone would never reach the video.

The formula card

A revision sheet of **every formula the question uses** — shown near the end of the video and as a slide in the carousel, the card a student screenshots before an exam. Each formula gets its own card: an icon, its number and name, the formula **typeset as in a textbook** (fractions stacked, √ with its bar, Vrms, ω, φ), a line saying what the symbols mean, and a **small graph** of what it says.

- **Write the formula card** has Gemini write it from the question, its answer and its working — text only, free tier. The first time, it also puts the sheet into the video. **Start one by hand** makes an empty card instead.
- **Every card is editable**, and the formula is typeset beside its box as you type. Formulas are written in LaTeX, the notation Gemini writes most reliably: `V_{rms} = \frac{V_0}{\sqrt{2}}`, `P = V_{rms} I_{rms} \cos\phi`, `\omega = 2\pi f`. Anything it does not know is shown as written, never dropped.
- **The graphs**: a sine wave, a sine with its RMS level, a rotating phasor, voltage and current out of phase, the power triangle, a straight line, a 1/x curve, and charging and discharging curves. In the video each one **draws**

itself in as its card arrives and then keeps moving — a dot runs along the wave, the phasor turns, the triangle's sides draw in turn. Choose *No graph* for a formula a picture would not help.

- **Icons** are found by name (*light bulb, ammeter, gear*). Change a name and press *Find the icons*.
- **Show it at the end of the video** adds one scene just before the outro, held long enough to read every card however short its spoken line. The other scenes **keep their voiceover** — only the new line needs recording, and step 4 offers to record just that one.

Up to six cards. In a portrait video they stack in one column; in landscape, and on a crowded carousel slide, two. The Doodle look writes the whole sheet in its marker. To see a sample without spending anything, `node --import ./tools/ts-resolve.mjs tools/formula-preview.mjs` renders the six AC formulas in several looks, both shapes and as the slide — add `--video` for clips.

The worked solution. The same Gemini request also writes the question's own working, as a teacher writes it on the board: the formula, the numbers put in, each step, the answer with its unit (`V_{rms} = \frac{V_0}{\sqrt{2}}`, `= \frac{325}{\sqrt{2}}`, `= 230\,V`). **Write it in, in the explanation** shows it in the first explanation scene — a line at a time across the scene, the newest in the accent colour, the answer boxed (ringed in marker in Doodle). It takes the place of that scene's own picture, which comes back if you turn it off. Edit the lines freely; a unit after `\,` is set upright, as in print.

Resistances combining. When the question's circuit is plain series (one loop) or plain parallel (every load across the same two points), a switch here shows them **sliding together into one Req** in an explanation scene, with the formula above and the numbers below. Anything more tangled — a ladder, a bridge, a capacitor — gets no animation rather than a wrong one.

Other hooks. On the script's hook scene, 💡 **Other hooks** has Gemini write four more opening lines in different styles (a doubt, a bold claim, a surprising consequence, a challenge). Press one to use it — only that scene then needs its voice again. None may give the answer away; one that does is dropped. The thumbnail maker has the same for its headline: 💡 **Headline ideas**.

On an explainer, a scene that draws a layout shows a 🖼 **Drawn on screen** row listing every label in it — and on a motion scene, every cue that fires an animation. A label marked ⚠ is one your narration never says — the reveals are timed by matching the spoken words against those labels, so an unmentioned label can only appear on a guess. Mention it in that scene's narration, in the order shown, and the warning clears.

Step 4 — Voice

Pick a voice from your own ElevenLabs account and press **Hear this voice** to sample it. If the list is empty, add any voice from the Voice Library and reload.

The cost is shown before you spend anything. Press **Make the voiceover**.

> **This is where sync happens.** Every clip is decoded in your browser and measured for real — exact length, where sound starts, where it stops. Each scene is then made exactly as long as its own clip, always rounding *up*, so a line can never be cut off. The subtitles are nudged by the few milliseconds of silence every MP3 carries at its start, which lands the highlight on the right syllable.

Changed a line after recording? Step 4 notices which scenes have lost their recording and puts **Record the changed scene** first — it records only those, for only their characters, and drops the new clips in beside the ones you kept. *Continue* waits until they are done, because a scene with no voice has no length. **Record everything again** is still there if you want a fresh take throughout.

The options light up as they are read, because the narration reads them in order.

Open **Show the sync report** to see the numbers per scene. If a line ever sounds clipped, turn off **Trim trailing silence** in step 5.

Step 5 — Look

Everything here is instant, free, and never touches the voiceover.

- **Dark or light**, and the layouts: **Simple** (clean), **Elegant** (serif, documentary), **Nerdy** (terminal green on graph paper), **Flashy** (loud, best in a feed), plus one for every design system you have brought in from Claude Design — **Organic** so far. See Looks from Claude Design.
- **Where the text sits** — *Match the look*, *Centred* or *Flush left*. Only blocks of text move; labels stay under the thing they name.
- **Highlight colour**, **thinking time** (3–5s), **breathing room**.
- **Show the spoken words** — the read-along text. Leave it on; most people watch on mute.
- **Draw the diagrams** — on the explanation and outro scenes, and a **setup diagram on the question scene**: the circuit, the apparatus or the geometry being asked about. Anything that could hint at the answer is stripped from that one automatically — no charts, no pies, no comparison panels, and no caption. As well as static ones (a formula box, comparison bars, a side-by-side panel, an icon) Gemini can choose a **live animation** from a fixed library of ten: wave interference, a travelling wave, orbits, a projectile arc, a pendulum, a vector field, spreading particles, a graph being drawn, an atom, light refracting — plus six built for electrical and power work: a **circuit** (any arrangement — see below), a **phasor diagram** and power triangle, an AC **waveform** (phase shift, rectified, PWM), a **block flow** that lights up stage by stage (boiler → turbine → condenser → pump), a **transformer** with turns ratio, and a **pie** for a fuel mix or a loss breakdown. There are now **over two hundred** of these — enough that every sub-topic the tool offers has one that fits — covering mechanics (levers, pulleys, gears, springs, collisions, friction, torque), light and sound (reflection, lenses, prisms, the Doppler effect), heat and fluids, chemistry (molecules, pH, titration, electrolysis, reaction profiles), biology (cells, DNA, neurons, the heart, photosynthesis, food chains), maths and geometry, data charts, earth and space, and sixteen built for aptitude and reasoning — a **Venn** diagram, a **clock face** with the angle between the hands, a **number line**, a **ratio bar**, a **seating arrangement**, a **family tree**, a **histogram**, a **logic grid**, **cube nets**, **dice**, **paper folding**, **mirror images** and more. Gemini is told not to use the same one twice in a row, and the tool drops it if it does anyway.
- **Cuts and text** — how one scene becomes the next, and how the words arrive. See Cuts, text and the finishing layer.
- **Finishing layer** — grain, a vignette, cinema bars. Same section.
- **Drift topic symbols** — faint themed emoji behind everything.
- **Moving backdrop** — one of thirty slow animations under the whole video. See Moving backdrops below.
- **Backdrop photos** — press **Find backdrop photos** and it searches Pexels and NASA per scene, using a search term Gemini wrote for that scene. **Nothing is applied for you**: a photo library will cheerfully return a beach for "gravity". Click the ones that fit, skip the rest.
- **Drawn backdrops** — with billing on your Gemini key (or an ElevenLabs Pro plan), each scene also gets a **Draw** button that makes a picture instead of finding one. Scenes where *no honest photo exists* get a drawing prompt written for them and can be drawn and attached in one go. See Drawing your own backdrops.
- **Sound** — three built-in music beds (calm, tense, upbeat), or load your own file. The music **ducks automatically** under the narration. Effects: a countdown tick, an option whoosh, an answer chime, and a sweep between scenes.

Circuits are described, not chosen from a list. The circuit diagram used to offer two shapes, *series* and *parallel* — so a question about two resistors in parallel wired in series with a third got whichever of the two was closer, and the picture disagreed with the words. A picture that contradicts the question is worse than no picture: the

words get checked and the picture gets believed.

It now takes the **network itself**, written as a short line where `+` is series and `|` is parallel — and **each part is drawn as what it is**. That second half is new. The line used to draw every part as a resistor on a battery, whatever the scene was about, so a series RLC circuit came out as three resistors of 10, 20 and 30 Ω and a transmission line's capacitance as two 50 Ω resistors across a cell. Five of its eight uses in real videos were wrong for that one reason.

Written	Drawn
`R + L + C`, supply `Vs`, AC	a resistor, a coil and a capacitor across an AC source — a schematic, with no values invented
`Rm=99Ω | Rsh=1Ω?`, supply `100 mA`	a meter resistance and its shunt hanging from a current source, the shunt shown as **?** until the answer
`Req=0.5Ω + Xeq=j1.41Ω`, supply `30 V`, AC	a resistance and an inductive reactance (`-j5Ω` is a capacitive one)
`Ra=0.5Ω + Eb=210V`, supply `220 V`	an armature resistance and a back EMF, opposing the supply
`L + (C | Load)`, supply `Vs`, AC	a line: its inductance along the top, its capacitance and its load across
`4Ω + (6Ω | 3Ω)`, no supply	a network between two terminals **A** and **B**

A part is what its **unit** says — Ω a resistor, mH an inductor, µF a capacitor, V an EMF — or, with no value, what its **name** starts with, the way a circuit diagram letters its parts: R, L, C, X, E. There are also a lamp, a switch (open or closed), a diode, a thyristor, a load, and the meters A, V and G. A part that cannot be read is **refused, never guessed**, and a circuit must say whether it is AC or DC, so nothing is drawn on a battery by default.

It is solved, and can be checked. When every part has a value the circuit is worked out by the same nodal analysis as the question's own figure, and current flows round it after the answer. The script may also state one fact about it — *I(Rm) = 1 mA*, *I = 20 A*, *R = 6 Ω* — which is then **checked against the solved circuit** and shown underneath. A circuit that does not give that value is not drawn: it is not the circuit in the question. Step 3 says so under *Asked for, and not drawn*, with what it actually works out to.

Parallel binds tighter than series, as it does on paper, so `R1 + R2 | R3` is R1 in series with the pair. A circuit that ends in parallel branches is drawn as a **ladder** — each branch a rung between two rails — which is how a shunt, a divider or a line is drawn in a textbook. A bridge does not reduce to series and parallel and is a separate figure.

To see the line drawn without making a video: `node --import ./tools/ts-resolve.mjs tools/circuit-line-preview.mjs` renders a dozen circuits, each on a question scene and on an explanation, into `out\frames\circuit-line`.

The standard characteristics are drawn from their equations. A script about induction-motor torque once asked for "a graph" and got an upward curve; one about a thyristor's holding current got a diode. The library had the right curve for neither, so the nearest shape was used — and a curve of the wrong shape is a wrong statement. These now have their own drawings, each computed from the equation a textbook gives, with the landmarks an examiner asks about marked and named:

Characteristic	What it shows
Torque against slip	Starting torque, maximum torque and the slip it occurs at, $s = R_2/X_2$. A second form draws the family as rotor resistance rises: the peak keeps its height and moves to standstill at $R_2 = X_2$.

Characteristic	What it shows
Series resonance	The peak at f_0, the half-power points at 0.707 of it, and the bandwidth f_0/Q. A second form sets a high-Q curve against a low-Q one.
Thyristor	Forward blocking to the breakover voltage, conduction, the latching current above the holding current, reverse blocking and breakdown.
Hysteresis loop	Saturation, retentivity and coercive force; or a narrow soft-iron loop against a wide hard-steel one.
Magnetisation curve	The air-gap line, the knee, saturation, and the residual voltage it starts from.
Transformer efficiency	Efficiency against load, with iron loss and copper loss crossing under its maximum.
V-curves	Armature current against field current: unity power factor at the foot, lagging and leading either side.
DC motor speed against torque	Shunt, series and compound through the same full-load point.
Power over time	Steady power as a rectangle whose area is the energy — one load, or two set together. The energy is worked out, not written.
Phasor diagram	Two to four phasors, each with its own length and angle, drawn from one point or **added head to tail**. When they are added, the last one is checked against the sum of the others: a sending-end voltage drawn larger than the receiving end it should be smaller than is refused, with what the sum really comes to.

None of them carries an invented number: they are shapes and relationships, true of every machine of the kind. `node --import ./tools/ts-resolve.mjs tools/curve-preview.mjs` draws them all into `out\frames\curves`, to check each against the textbook.

Each video is offered only the diagrams for its subject. The library has over two hundred, and all of them used to be described in every request — nine tenths of everything the model was told, so a script on transformer testing was offered a DNA helix and a seating puzzle. A model given two hundred choices picks the nearest thing more often than the right one. An electrical video is now shown fewer than ninety: the electrical diagrams, the mathematics an engineering paper leans on, and the ones every subject shares (a flowchart, a classification tree, a chart of its own numbers). Aptitude, physics, chemistry and the rest each have their own shelf; the science and computer sections of the aptitude paper are given science and computing, not puzzles; and a subject without a shelf — general knowledge, history — still sees the whole library. It goes by the subject you picked, never by a word in the topic. **See exactly what will be sent** on step 2 shows the list a video is given.

Play the phone preview before rendering. Fixing something here takes a second; after a render it takes minutes.

Step 6 — Export

Choose a quality (**Normal** is right almost always) and press **Render the video**.

> **The first render is the slow one.** The very first time, the tool downloads a rendering browser of about 150 MB. It can look like nothing is happening for several minutes. It only ever happens once per computer.

A 45-second video takes one to three minutes. You get a player, a download button, and the file is saved into the `out` folder automatically.

A render cut short by closing the helper is not lost. Next time, step 6 lists it under *A render was interrupted* with **Render again**, which runs it exactly as it was — same script, same voice, same look — even if you have changed the video since.

The video library

Press **Library** in the top bar. Every video you make is there, newest first, **saved as you work** — nothing to press. Each shows its question, when it was last touched, **what it cost** (pictures in dollars, voice in characters), how often it was rendered, a link to the last render, and where it was published.

- **Open** brings a video back — its script, look, formula card, drawings and voice — onto the Script step, to fix, render again or publish. What you were working on is already saved.
- **Start a new video** begins a fresh one. So does generating a new question.
- **Search** by question or topic; ✕ deletes a video from the list (its MP4 files are kept).

The list lives in the `library` folder and is **committed with the code**, so after a `git pull` your other computer has it too. The voice clips and drawings are not committed — they are large — so a video opened there tells you what is missing, and it is simply made again: the Voice step records only the missing scenes, and the Look step draws only the missing pictures.

🌐 **Translate** on any video makes a copy of it in Hindi, Kannada, Tamil, Telugu, Marathi, Bengali, Malayalam or Gujarati — the question, options, script and formula card translated as an Indian engineering teacher would say them; the maths, the circuit, the answer and the look unchanged. The copy is a new video in the library and opens on the Script step to check; then record its voice. **Choose an ElevenLabs model that speaks the language** on the Voice step — not every model covers every Indian language, so check ElevenLabs' list for Kannada in particular. In the Doodle look the Indian scripts are set in a regular font, not the handwriting, which covers English only.

The planner and batches

Press **Planner** in the top bar.

Plan. Choose an exam, optionally a focus (*Electrical Machines only*), and how many questions. Gemini lays out the exam's syllabus as units and picks a run of questions across it — each a topic and the angle it should test, spread so no unit takes more than three in a row. **Every topic already in your library is left out**, and checked again after Gemini answers. Free tier. *Plan more* adds to the same exam's list later; plans are saved in `library/plans` and travel with the library.

Make one by hand. *Make by hand* fills the Topic step with that question — exam, subject, topic and the angle — and takes you there. When its question is generated, the item is ticked off and linked to its video.

Make several at once. Tick questions (or *Select the next 5*) and press **Make N videos**. Each is written, **checked by DeepSeek** if you have a key, has **its pictures checked** and the wrong ones taken out, is given its formula card and worked solution, directed for the engineer in the Doodle look — with the pictures beside him if you tick *Draw what stands beside the engineer* — voiced, measured and rendered, one after another, and saved to the library. It uses your current look, voice and topic settings; choose a voice on the Voice step first.

- **A video DeepSeek disputes stops before its voice** and stays in the library, marked to check — no batch should publish past a wrong answer. So does one whose **pictures could not be checked** (a rate limit, say), and anything that fails; the rest carry on.
- **It runs in this tab.** Keep it open and the computer awake until it finishes — overnight is fine. *Stop after this video* ends it cleanly.
- What each video cost is in its library entry, as for any other.

Rehearse the Gemini features (on step 1) runs a sample question through the formula card and the doodle directing and checks every reply has the shape the video needs — poses from the list, formulas that typeset, graphs that exist. Free tier; nothing is drawn or voiced. Worth a press after any update, before a real video depends on it.

Step 7 — Publish

Fill in **Channel or site** first — it goes into the description and onto every carousel slide.

Press **Write the metadata** and Gemini writes it from the finished video, for three platforms at once, in three tabs:

Tab	What you get
▶ **YouTube**	**Several title options** to choose from, a description, tags, hashtags, suggested thumbnail text and a pinned comment.
📷 **Instagram**	A Reel caption with its hashtags, and alt text. See Instagram and Facebook.
👍 **Facebook**	A title, a short description with hashtags, and tags.

Every box has a copy button and is already inside that platform's limits. For exam-prep videos the title and first line lead with the exam name, subject and topic, because that is what people type into search.

Further down the same step: the thumbnail, the carousel post, and the upload kit, which packs all of it into one zip.

Diagrams that are worked out, not guessed

A diagram that shows the wrong number is worse than no diagram, so for the question itself the tool does not let the model draw numbers. The model only *describes* the setup — which components, which values the question states — and the tool **works out** everything else itself and draws it.

Nine kinds are built:

Kind	Draws
Junction	Currents meeting at a node (Kirchhoff's current law).
Circuit	DC and AC circuits, solved for every current and voltage.
AC	Phasors and waveforms, with the power shaded.
Power triangle	P, Q, S and power factor, including power-factor correction.
Three-phase	Star and delta, line and phase values.
Transformer	Ratios, currents, EMF, efficiency and regulation.
Machine	Induction and synchronous speed and slip, DC back EMF and generated EMF.
Graph	Curves from formulas, with roots, peaks, slopes and crossings worked out.
Bar model	Ratio, percentage, profit and discount, interest, time and work, mixtures, replacement, averages.

On the **question scene** anything the question does not state is shown as **?**, so the diagram never gives the answer away. Graphs never appear on the question scene at all — a marked point on a curve *is* the answer. On

the explanation the same diagram comes back with every value filled in.

The card on step 3 tells you what happened:

Card	Meaning
✓ **The diagram agrees with the answer**	The worked-out value matches the marked option. A genuine second check.
✕ **The diagram does not agree with the answer**	They differ, so the diagram was **left out** rather than show one number while the voice says another. Check the question — one of the two is wrong.
⚠ **The diagram was not drawn**	The description could not be drawn honestly — the card lists why. Generating again usually fixes it. One common reason: the diagram **marked nothing for the video to work out**, so it could not be checked, and a diagram like that is usually a picture of a different question, with values the question never gave.
The diagram is drawn but not checked	The answer has no number to compare, e.g. a word answer. Look it over yourself.

> The cards say "circuit" whichever kind of diagram it is — a graph or a bar model gets the same wording. The check itself is the right one for that kind.

A conceptual question gets no diagram, on purpose. "*How many times does alternating current pass through zero in a cycle?*" has no particular circuit behind it, so there is nothing honest to draw. The model is told to ask for no figure in that case, and any figure that marks no unknown is dropped rather than drawn — that is exactly where invented values used to creep in, such as a phase lag nobody mentioned.

Topics without one of the nine kinds still get the illustrated diagrams and animations from step 5.

Circuits come alive at the answer. Once the answer is out, a circuit diagram switches on: current flows round every wire as a train of beads in the look's accent colour, and every lamp lights up. Both come from the same solved circuit as the answer, so they are exact:

- **Faster beads mean more current.** Each wire runs at its share of the biggest current in the circuit — in two parallel branches of 6 Ω and 3 Ω, the 3 Ω branch visibly runs twice as fast. Speeds are relative rather than so many per ampere, so a milliamp circuit still moves and a twenty-amp one never runs so fast it seems to go backwards.
- **Brighter lamps take more power.** A lamp on 48 W shines more than one on 24 W beside it. A lamp shorted out, or in a branch carrying nothing, stays dark.
- **The beads follow conventional current**, out of the + terminal and round — the same way the question's arrows and every textbook draw it.
- **AC rocks back and forth** instead of drifting, each branch in its own phase, so a capacitor's current visibly leads a resistor's.

Nothing moves before the answer. In the question scene the circuit is the still drawing it always was: how fast the current runs and how bright each lamp is are exactly what these questions ask, and showing them would answer the question before the countdown.

To see it without making a video, `node --import ./tools/ts-resolve.mjs tools/circuit-preview.mjs` renders a lamp circuit and an AC circuit, in several looks and both shapes, into `stills/` — add `--video` for clips.

Pictures that are checked, and the test bench

A wrong picture does more harm than a wrong sentence, because a viewer believes a diagram more readily. The diagram of the question itself is worked out and checked against the answer (the section above). Every *other* diagram — the animated sketches, the bar charts, the side-by-side cards — now has to get past three things before it reaches a video. The price is fewer pictures: a scene with none is the tool working, not failing.

1. The rules: nothing is drawn that was not stated. These need no model and cannot be talked round. A picture stopped by one leaves its scene bare, and step 3 lists it under *Asked for, and not drawn* with the reason.

Rule	What it stops
Everything a sketch draws must be stated.	Each sketch has knobs — an angle, a ratio, two labels, a list. One left out used to be drawn at its default as though it were a fact: a 60° lag nobody mentioned, a power factor of 0.81, a meter reading 12.4 V. Now a sketch missing anything it lists is not drawn, and neither is a list sketch with no list, which used to come out as a blank scene.
Every number needs a source.	A bar value, a circuit component or an angle must be one the video gives — in the question, the options, the working, a formula or a spoken line — or one step from those (5 + 3, half of it, volts to millivolts). A series RLC drawn as resistors of 10, 20 and 30 Ω, or *110 kV against 145 kV* for a question with no numbers in it, goes.
A label is drawn whole.	Sketches cut long labels to fit, silently: *Keep dependent sources* became *Keep dependent sourc*, and a meter told to show *Measure Response* displayed the reading *Measure*. A label that would be cut stops the sketch.
The question scene never shows the answer.	A shunt question was illustrated, during the question, with the meter beside its 0.099 Ω shunt.

2. The picture check: is it true of the line it sits on? A picture can be fully stated and still wrong — a diode's curve under a line about a thyristor. That is a judgement, so it is asked of a reviewer whose only job is to make it. For each diagram it is given the question, the answer, the spoken line, the picture in words and **the frame as it will be rendered**, and gives one verdict:

Verdict	Means	What happens
Right	It shows what the line says, and nothing on it is false.	Stays.
Wrong	Something on it is false for this question: the wrong component, the wrong relationship, an invented value.	Taken out.
Beside the point	Not false, but only about the same subject — a spanner for *load torque*.	Taken out.
Not sure	The reviewer could not tell.	Stays, flagged for you.

Gemini looks at the frames; DeepSeek, when its key is there, reads the same pictures in words. **A picture is taken out when the two agree it should go.** On the first real video checked, every picture both objected to was wrong, and every picture only one objected to was sound — a reviewer alone is quick to call a plain picture "generic". Where they differ the picture stays, marked **Not sure**, with what each said, for you to judge. With only one reviewer, it takes out what it calls wrong and leaves what it calls beside the point to you. It runs on its own when a script arrives on step 3, and in every batch before the formula card. The first check after starting the studio takes about a minute while the video engine warms up; after that, a few seconds a picture. Free on Gemini's free tier, and a fraction of a cent of DeepSeek. **Put it back** overrules it for good — that picture is not taken out again.

3. The test bench: is any of this working? Press **Accuracy** in the top bar. The bench is a fixed set of questions, the pictures made for them, and *your* mark on each one — **Right**, **Wrong** or **Beside the point**. Your marks are the truth; the rules and the reviewer are being measured against them.

- **Add my library videos** puts every video you have made on the bench. Free, no keys.
- **Add questions from the syllabus** writes scripts for a spread of topics across the electrical syllabus, a calculation and a concept in turn, and checks their pictures. Nothing is voiced, drawn or rendered, so it costs nothing on the free tier; it waits between questions to stay inside it.
- **Mark** each picture from its frame (click it to see it full size) and say what is wrong in the box beside the buttons. That note is what says what to fix next.
- **A dashed button is a suggestion.** Where Claude has already been through a picture it leaves a suggested mark and its reason. It counts for nothing until you press a mark yourself.

The tiles are the score, on your marks alone:

Tile	Reads
Right, of what a viewer sees	Of the pictures a video made today would show, how many you marked right. The number that matters.
Wrong pictures still shown	Marked wrong or beside the point, and neither a rule nor the reviewer stops it. **This has to be zero.**
Wrong pictures stopped	Of the bad ones, how many are stopped, and by which of the two.
Right pictures kept	Of the good ones, how many survive. The cost of being strict.
Right, before any check	What Gemini asks for, unfiltered — the raw material.

A picture you mark wrong stays on the bench for good. It is the standing test that whatever let it through no longer does: change a rule or the reviewer's instructions, press **Check again**, and the tiles say whether it helped. The filters find the ones worth a look — *Wrong, and still shown* is the to-do list; *Reviewer disagrees with me* is where the reviewer needs teaching.

Cases and marks are saved in the `bench` folder and are committed with the code, so they reach your other computer with a push. The frames are drawn into `out\bench` and are not; **Draw and check** makes them again.

From a terminal, without the app:

```
npm run bench
node tools/picture-replay.mjs
node --import ./tools/ts-resolve.mjs tools/sketch-defaults.mjs
```

The first prints the score. The second runs the rules over every video in the library and lists what they would stop today. The third shows what each sketch puts on screen when it is told nothing — the list a new sketch must not add a number to.

Instagram and Facebook

The same Reel goes to Instagram and Facebook, but they are searched differently from YouTube, so the text is written for each rather than copied across.

Instagram

- **The caption is what search reads.** Instagram's search looks at the words of the caption, so the topic and exam are written into the sentences themselves.
- **The first line is the hook.** Only about 125 characters show before *more*; the page warns you if the first line is longer.
- **Five hashtags, no more.** Since December 2025 Instagram ignores every hashtag past the fifth — and it counts hashtags in comments too. The tool caps the list at five, takes any that slipped into the caption back out, and drops YouTube-only tags like #Shorts. Do not add more in a first comment.
- **Alt text** — paste it under *Advanced settings → Accessibility → Write alt text*. It helps search as well as screen readers.

Copy **Caption with hashtags** — it is one block, hashtags already at the end.

Facebook

Facebook shows a line or two before *See more*, so the description is short, with one to three hashtags. The **title** is for a normal video upload — a Reel has no title field. **Tags** go in the video Tags field where the upload form offers one.

> Metadata written before these tabs existed shows "*No Instagram text yet*". Press **Write it again** to get it.

Making a thumbnail

Step 7 has a **Thumbnail** section. It renders an image in the same colours as the video, on your own machine. The words are always free to render as often as you like; only a picture painted by Gemini costs anything.

Design it with Gemini

Press ✨ **Design with Gemini** and it does the whole design in one go:

1. **Gemini reads the title you picked and the description** and writes a short, scroll-stopping headline of two to five words, picks the one word to colour, the layout, a corner tag and a symbol.
2. **Gemini's image model paints a picture** for behind it, composed to leave the left side (16:9) or the top (9:16) calm for the words — with **no lettering in the picture at all**.
3. **The words are set on top as real type**, so they are always spelled right and in the video's fonts. Image models still misspell words; a thumbnail reading "Tranformer" is worse than none.

All the fields below fill in, and you can change any of them and press **Make it again** without paying for the picture again. **The picture** box shows what was painted — edit it and press 🩹 **Repaint the picture**, or untick **Use the picture** to go back to the plain backdrop.

It never gives a quiz away. Gemini is not told which option is correct, and a headline or figure that repeats the correct answer anyway is replaced — a note tells you when that happened.

What it costs. The headline design is a normal text request (free tier). The picture needs **billing on your Gemini key**, about 3c to 13c depending on the **Picture model**. Untick **Paint a background picture** and you get the design without the picture, for free.

> The button needs a Gemini key on step 1. It works best after **Write the metadata**, because it builds from the title and description you chose; before that it works from the question.

Designing it yourself

Pick the shape first. It defaults to the shape of the video you just made.

Shape	Size	For
16:9	1280 × 720	The YouTube cover image on a normal video.
9:16	1080 × 1920	Shorts, Reels and TikTok.

The layouts adapt rather than being cropped: in 9:16 the **split** symbol moves above the text and the **question** mark sits on its own line, because a narrow frame cut into two columns leaves both too thin to carry anything. Everything is kept to the middle of a portrait frame on purpose — the apps put their own title, channel name and buttons over the top and bottom.

Layout	Use it when
Statement	One bold claim. Works for most videos.
Question	A huge **?** beside the text. Classic quiz thumbnail.
Number	The answer is a figure — 8,760 · 60% · 50 Hz. The strongest of the four when it fits.
Split	Text and one big emoji — side by side in 16:9, stacked in 9:16.

Wrap a word in `*asterisks*` to colour it with your accent — `hits the ground *first*` puts *first* in the highlight colour. It is the single thing that makes a thumbnail read as designed rather than typed.

If you wrote the title and tags first, the **thumbnail text** Gemini suggested is filled in for you.

> **Judge it in the small box.** Two previews appear: one at the width the thing is really seen at — about **320** pixels for a 16:9 row, about **200** for a portrait shelf — and one full size. The small one is the honest test. If you cannot read it at a glance there, cut words out — six or fewer is the target, and the field warns you above that. Shrinking the type to fit more in is what makes thumbnails invisible.

The PNG is saved into `out\` next to your videos. Press **Save the PNG** to put it wherever you like.

A carousel post

The Reel gets watched once; a carousel gets **saved** and come back to before an exam. Step 7 turns the same question into a swipeable post of **square 1080 × 1080 slides** for Instagram and Facebook, in the video's own theme. It renders on your machine and costs nothing.

Press **Make the carousel**. The slides come in the order a reader wants them:

Slide	Shows
Question	The question and its options A–D, with *Swipe for the answer* →. A question with a diagram gets the diagram here, and the options move to their own slide. A very long question does the same.
Answer	All the options — the right one in green with a ✓, the others struck through — and the answer line.
Worked out	The question's diagram with every value filled in, when it has one.

Slide	Shows
Why	The explanation as numbered steps, over as many slides as it needs (*Why · 1 of 2*).
Formulas	The formula card — every formula the question used, typeset, with its graph — when you have written one.
Did you know?	The fun fact, *Save this for revision · Follow for one every day*, and your channel name.

Every slide carries a counter (*2 / 5*) and your channel along the bottom, so the post reads as one set in a profile grid. Text is sized to fit the square, and a post never exceeds Instagram's limit of twenty slides. Explainers without options get the question, the why slides and the follow slide.

Below the button: a strip of all the slides and a larger view with **Previous / Next** to check each one, **⬇ This slide** to save one, and **⬇ Save all slides (.zip)**. The zip holds:

```
gravity-carousel.zip
├── slide-01.png … slide-04.png
├── caption-instagram.txt   the Instagram caption + a "Swipe   " line + the hashtags
├── caption-facebook.txt    the same for Facebook
└── HOW-TO-POST.txt
```

Posting it

- **Instagram: + → Post**, tap *select multiple*, pick the slides **in order**, keep the square crop, and paste `caption-instagram.txt`.
- **Facebook: Photo/video**, select every slide in order, paste `caption-facebook.txt`.

> Write the metadata first if you want the captions in the zip — the slides themselves do not need it. Make the carousel before packing the upload kit and it goes into the kit too.

The upload kit

The boxes on step 7 are gone the moment you close the tab, and the upload usually happens later — on another day, or from another machine. **Pack the upload kit** puts all of it in one zip beside your video, along with the thumbnail.

```
how-does-a-fish-get-past-a-dam-upload-kit.zip
├── UPLOAD.txt           the whole form, in order, with the character counts checked
├── title.txt            one field per file, for fast copy-paste
├── description.txt      complete and paste-ready — chapters and credits already in it
├── tags.txt
├── hashtags.txt
├── pinned-comment.txt
├── chapters.txt
├── credits.txt
├── instagram.txt        the Reel caption with hashtags, and the alt text
├── facebook.txt         title, description with hashtags, and tags
├── metadata.json        if you ever script the upload
├── thumbnail.png
└── carousel\            the carousel slides, both captions and HOW-TO-POST.txt
```

`instagram.txt`, `facebook.txt` and `carousel\` are only there when you wrote the metadata and made the carousel before packing. If an icon in the video needs a credit, that line is added to the Instagram and Facebook captions as well as the YouTube description.

Open **UPLOAD.txt** first. It walks the YouTube form field by field, counts every character against the real limit, and shouts if the title is over 100 — silently trimming it would hand you something YouTube cuts off mid-word, which you would only notice after publishing.

Two things in the kit cannot be copied off the page, because only the tool knows them.

Chapters are real timestamps, computed from the scene timings that produced the video. Nobody can type these accurately afterwards. They are already inside `description.txt`, so pasting that one block gets you the chapter bar under the scrubber for free.

You do not always get them, and that is deliberate. YouTube ignores a chapter list unless it starts at `0:00`, has **three or more** marks, and none is **under ten seconds** — and it does not tell you it has ignored it. So short scenes are merged, and a video that cannot have a valid list gets none rather than a broken one. Shorts never get chapters.

Credits lists what actually ended up in this video and what each thing asks for in return:

What is in the video	What it asks for
Icons	Named per set. Anything wanting a credit is flagged, and the exact line to paste is written for you — it is already in the description.
Photos	Any stock you picked, with its credit line.
Music	Always yours. It is synthesised here from scratch — not sampled, not licensed from anyone, so no copyright claim is possible.
Narration	Flagged as a synthetic voice, since some platforms want that disclosed.

Pack it again any time — after changing the title, or after making a thumbnail or a carousel. It rebuilds from whatever is on the page at that moment.

Using Claude instead of Gemini

Add a **Claude API key** on step 1 and a **Who writes it** choice appears on step 2. Pick **Claude** and the question or the storyboard is written by Claude instead; pick **Gemini** and nothing changes.

The choice only appears once a Claude key is present — a button that could only fail is worse than no button. Remove the key later and the tool quietly goes back to Gemini; your preference is remembered, so pasting the key back returns you to Claude.

The **Claude model** dropdown replaces the Gemini one and is built from your own key, newest and most capable first — Opus above Sonnet above Haiku. Opus is the default.

The rest of the tool does not care which one wrote it. The review gate, the voice, the diagrams, the render — all identical.

> **Two steps still use Gemini, whichever you pick. What is trending now** needs live web search, and the title and tags on step 7 are written by Gemini. Keep your Gemini key even if you write with Claude; step 2 reminds you.

On cost — read this before switching. Gemini has a generous free tier. Claude has **none**: you are billed from the first request, and you have to add a payment method before any key will work.

Rough cost of one video's worth of writing, at the published rates:

	Claude Opus 5	Claude Sonnet 5
One quiz	~9c	~4c
One five-minute storyboard	~35c	~14c

Estimates, not quotes — the real figure moves with how long the script is and how much the model thinks. Two things follow from them. **Output dominates**, so the storyboard costs far more than the quiz and the model choice matters more than anything else you can change. And **a day of making shorts on Opus is a few dollars, not a few cents** — set a spend limit in the Anthropic console so a runaway loop cannot surprise you.

If cost matters more than the last few percent of quality, Sonnet is the sensible default here.

Writing the script yourself

You do not have to let Gemini write it. On step 2, set the **format** and the **target length**, then press ✍ **Write it myself** instead of Generate.

That drops you straight into step 3 with an empty script of the right shape — the correct beats in the correct order, and about the right number of them for the length you chose. A 180-second explainer gives you 18 scenes to fill in; a 45-second quiz gives you the question, the four options, the countdown, the answer and a few explanation cards.

It costs nothing and needs no Gemini key. Credits are only ever spent on step 4, the voice.

In step 3 every scene has a **kind** dropdown, ↑ ↓ to reorder and ✕ to delete, and there is an **Add a scene** button underneath. A bar shows how many seconds you have written against the length you asked for.

> **One thing that is not a slider.** You cannot set how long a scene lasts. A scene lasts exactly as long as its recorded narration — that is the whole reason the words on screen stay locked to the voice. The bar in step 3 is an estimate at the voice's own speaking pace; the real length arrives with the voiceover in step 4.

Real artwork in every layout

A diagram box used to hold a label and, at best, one emoji. Now every box, every process step and every grid cell can carry a **drawing of the thing it names** — pulled from the same open library of about 200,000 icons the moving scenes use.

The storyboard writes a plain English noun — `boiler`, `turbine`, `fish`, `battery` — and the tool finds it. The drawing arrives in **your theme's colours**, so it never looks pasted in from somewhere else, and it brightens as the narration reaches it.

This is why a layout stops looking like a row of empty rectangles, and it costs nothing: the drawings are fetched once when the script is written and travel inside it from then on.

The storyboard is told to name a picture wherever a **real object** is on screen, and to leave it out for an abstract idea — there is no useful drawing of *efficiency*, and a wrong picture is worse than none. Where nothing matches, the layout falls back to the emoji or to plain text, so a missing icon costs you one shape rather than the scene.

Nothing is ever completely still. Things that hold themselves up in a fluid — a fish, a bird, a balloon — keep swimming on the spot. Flames flicker. Anything that turns keeps turning. It is small enough that you will not consciously see it and large enough that its absence is what makes a frozen sticker look like a frozen sticker.

Animation comes from your verbs

Every explainer scene used to animate once as it appeared and then hold perfectly still. On a twenty-second scene that is about one second of movement and nineteen of a screenshot — which is what makes a video feel flat even when every frame is correct.

Now the narration drives it. **The words you write are the animation.** When the voice says *flows*, something flows, at that moment. Say *spins* and something turns. Say *escapes* and it bursts outward. Nothing is authored, nothing is scheduled — it is read off the script.

Say	You get
flows, pours, travels, carries, circulates	particles crossing the frame
rises, climbs, grows, increases, expands	an upward drift
falls, drops, sinks, decreases, collapses	a downward drift
spins, rotates, revolves, orbits, turbine	soft arcs turning behind it
heats, burns, boils, combustion	a warm wash and rising haze
cools, freezes, condenses	a cold wash
collides, blocks, strikes, barrier, bounces	a shove and a flash
vibrates, oscillates, alternates, resonates	standing waves
voltage, current, charge, circuit, spark	electrical flecks
escapes, releases, bursts, erupts	particles thrown outward
glows, shines, brightens, radiates	a bloom
rain, droplets, liquid, leaks, floods	falling droplets

Step 3 shows an ✨ **Moves on these words** row under each scene, so you can see what a scene will do before you record anything. That row is also the control: change "*the steam goes into the turbine*" to "*the steam **flows** into the turbine*" and the scene gains a flow. **Write the verb you mean.**

Restraint is built in. At most four per scene, never two within about two seconds, always behind the text, and always low contrast. An effect is meant to be felt, not watched — the moment your eye goes to the particles instead of the diagram, it has failed. Words that are common English but rarely a real movement (*light*, *up*, *down*) are deliberately left out, because an effect firing on a sentence that did not mean it is worse than one that never fires.

Effects are aimed, not just timed. When the voice says "*the turbine spins*", the spin appears **on the turbine box** — not in the middle of the frame. The tool works out which item the narration has reached, from the same reveal order the panel itself uses, and fires the effect there. A flow passes through the row the active box is on; a bloom, a burst and a heat wash all centre on it.

That is most of the difference between an effect that explains something and one that is just decoration laid over the top.

In the Doodle look, the verbs are drawn on the words instead. Blurred particles are the wrong medium for a notebook page, so there the same words get a pen mark of their own, in the marker red, drawn on as the word is spoken — on every scene, the question and the hook included, not only the explainer layouts. *Flows* gets a wavy arrow underneath, *heats* gets steam over it, *spins* a turning arrow at its shoulder, *rises* and *falls* an arrow beside them, *sparks* a lightning bolt, *escapes* a burst of lines, *glows* a sparkle, *drips* falling drops, *cools* a snowflake, *collides* impact lines, and *vibrates* shake lines on both sides. The word itself moves a little with its mark: a rising word lifts, a shaken one shakes. At most two marks on a line, and never two of the same kind — an electrical question says *current* and *circuit* in one breath, and two bolts on one line would be decoration, not meaning.

Scenes now cross into each other. Each scene is held on screen a third of a second past its narration, so the outgoing and incoming ones overlap: one fades and slides out as the next fades and slides in, moving the same direction through the join. It used to be a hard cut with a five-frame flicker in front of it.

Three other things now move on their own, whatever you write:

- **Arrows carry something.** A diagram arrow used to draw itself once and freeze. Now a pulse travels down it for as long as the diagram is up — an arrow means *this goes to that*, and a still line is the one thing that cannot show it.
- **Process steps are joined by live connectors** rather than a `→` that only changes colour.
- **Every scene pushes in slowly** — about four percent over its length. Nobody consciously notices it; everybody feels the difference between a layout that was filmed and one that was screenshotted.
- **Things that should never be still, are not.** A fish holds station by swimming, a flame flickers, anything that turns keeps turning.

Scenes that move

Most explainer layouts show a **structure** and light parts of it up as the voice reaches them. One does something different: **motion** acts an event out.

A salmon swims at a dam. It is thrown back, twice. A fish ladder appears beside the dam, and the salmon climbs over it one step at a time. That is one scene, and the storyboard writes it in about eight lines.

Where the pictures come from. The storyboard names things in **plain English** — "fish", "dam", "turbine" — and the tool finds each one in an open icon library of about 200,000 shapes. They are fetched once when the script is written, cached, and then live **inside your script**. Nothing is downloaded while the video renders, so a render is as fast and as repeatable as any other, and works offline once the script exists.

Icons are drawn in your theme's own colours, so they never look pasted in from somewhere else.

Seven things can happen. The storyboard picks from a fixed list — it cannot invent an eighth:

Verb	What you see
appear	Fades and pops in. For something that arrives partway through.
move	Travels across and stops *beside* another thing.
blocked	Runs at something, is thrown back, tries again, gives up.
climb	Steps up and over something — the way through, once one exists.
pulse	Swells once, to say *this one, now*.
spin	Rotates on the spot.

Verb	What you see
exit	Drifts away and fades.

What times it. Nothing here runs on a stopwatch. Each beat carries a **cue** — a word or two from that scene's narration — and fires when the voice reaches it. The salmon is thrown back on the words "*wall of concrete*", not at 4.2 seconds. Rewrite the narration and the animation follows it.

That is also why a motion scene shows the same ⚠ warning as any other layout in step 3: a cue your narration no longer contains is a beat that can never fire. Put the words back and it clears.

When you get one. The storyboard is told to include one motion scene wherever the subject has a moment that genuinely moves — something blocked, carried, escaping, or finding a way past an obstacle — and to put it in the middle, where the mechanism is being explained. Two at most. On a subject that does not move, none is the right answer and you will get none.

To ask for one directly, put it in **Your own instructions** on step 2:

> *act out the mechanism with a moving scene*

The tool checks the storyboard and tells you. A motion scene fails quietly — a beat whose cue is missing still animates, just on a guess instead of on the voice — so the black PowerShell window says so the moment the script arrives, while regenerating is still free:

```
[generate] check: scene 4: the narration never says "sheer bulk", so those beats cannot fire on the voice.
[generate] check: scene 4: "collapses" is cued on the last few words, so that beat will barely be seen.
[generate] check: scene 4: "fish" and "dam" start almost on top of each other.
```

None of these break the video. They tell you it will be looser than it should be. Regenerating usually clears them; so does rewriting that scene's narration to contain the missing words.

One bad cue does not spoil the rest. Each beat is matched to the voice on its own, so if the model writes three good cues and one it never says, the three still land exactly where they should and only the fourth is guessed at.

Things travel at a speed, not for a time. A moving actor used to take the same 1.15 seconds whether it crossed a tenth of the frame or three quarters of it, so long journeys were flung and short ones stalled. Distance now sets the duration. Verbs that cover more ground than the straight line — a bounce goes in and back twice, a climb zig-zags up in steps — get proportionally longer.

Two beats never run at once. When the narration puts two cues close together, the first beat stops where it got to and the second carries on from there. They used to both keep going and add their movements together, which is the lurch that looked like the animation glitching.

Nothing is drawn over the words. Actors are kept inside a band that clears the scene title above and the caption band below, whatever coordinates the storyboard writes — including a beat that names an explicit destination.

If a video still feels busier than it should, step 5 has a **Narration effects** → **How strong** slider. Turn it down to calm everything the script triggers; at 0 the layouts hold still. It does not touch the moving scenes themselves — those are content, not decoration, and are turned off with **Draw the diagrams**.

> **Ask for it by asking for an event.** Motion is for something *happening* — being blocked, finding a way through, escaping, being carried. For a list, a comparison or a structure, the other layouts are better, and the storyboard is told to use them instead. Expect one or two motion scenes in a video, not six.

A note on credits. The tool prefers icon sets that ask for nothing in return — MIT and Apache licensed. If it ever has to fall back on a set that wants a credit, it says so in the black window when the script is written, and names the set to put in your description.

Moving backdrops

Step 5 has a **Moving backdrop** — a slow animation under everything, so a scene reads as produced rather than as text on a flat colour. Thirty of them, in ten families: drifting particles, flow fields, constellations, waves, orbits, a circuit-board grid, light beams, spirals, equaliser bars and contour maps.

Auto picks one from your subject: a circuit board for electronics, a lattice for transmission, equaliser bars for power generation, a galaxy for astronomy, contours for climate.

It is capped well below the text at every setting, so it can never make a caption hard to read. Turning it up makes it busier, not more distracting. It costs roughly **20% more render time** with the heaviest of them; set it to *None* if you want the fastest possible render.

Drawing your own backdrops

The keys you already have can also **draw** the picture behind a scene — your **Gemini** key with billing enabled, or your **ElevenLabs** key on a Pro plan. On step 5 every scene gets a **Draw** button next to its search box, beside the free Pexels and NASA search.

Why bother, when photos are free? Two reasons.

The first is shape. Stock libraries are full of wide photos taken for wide screens. Your short is tall, so the app has to cut a tall slice out of the middle of a wide picture — and whatever was happening at the left and right edges is simply gone. A drawn picture is made 9:16 (or 16:9 if that is what you chose) from the start, so nothing is thrown away.

The second is consistency. Twelve stock photos are twelve photographers, twelve lighting setups and twelve colour grades. Pick a **Look** — editorial photo, clean diagram, cinematic, or blueprint — and every scene you draw in that video comes back matching the others.

How to use it

1. Check the words in each scene's box. That is what gets drawn, so make it specific.
2. Pick a **Look** and an **Image model**. Flash models answer in a few seconds and cost the least; Pro is slower and sharper.
3. Press **Draw** on a scene. The picture appears among that scene's options with a purple **AI** tag and a dashed border.
4. Click it to actually use it — same as a photo. Not clicking costs you nothing further.
5. Not right? Press **Draw another**. The first one stays, so you can compare them side by side.

Which service draws it. There are two, and you pick with the tiles above the Draw buttons.

- **Google** *(recommended)* — uses your **Gemini** key. Needs **billing enabled** on that key's project; there is no free tier for image generation on any Google model. About **3p an image**, so roughly 25p for a whole video.

Writing the script stays free either way.

- **ElevenLabs** — uses your ElevenLabs key, and needs a **Pro plan**. The free and Starter tiers cannot draw at all.

Write what to draw. Each scene has a **prompt** link next to its Draw button. The search words above it were written for a *photo library* — two or three nouns — and an image model given two nouns draws two nouns. The prompt box is where you describe the picture instead.

- **draft one from the narration** turns what the scene actually says into a starting point. It drops the words that only work out loud — "*so*", "*you*", "*we*" — because an image cannot show the listener. On a hook or an outro it uses your topic instead: "*Most people get this wrong*" describes the viewer, not a picture.
- **Last sent** shows the exact prompt that went to the model, so a redraw is a correction rather than another guess.
- Leave it empty and the search words are used, exactly as before.

Whatever you write, the **Look** and the composition rule are still added on the end — so an edited prompt keeps the style match and keeps the middle of the frame clear for your captions.

Which model, and how to find out. The model dropdown is the biggest quality lever in this step, and it is impossible to feel one press at a time — by the time the second image arrives you are comparing it against a memory. **Compare models** draws the *same prompt* with all four at once and puts them side by side:

Model	Roughly
Flash Lite	3c — the default
Flash	4c — best at matching a reference
Flash 2.5	4c — the older model
Pro	13c — best quality

Click whichever you would keep: it becomes that scene's backdrop **and** the model the Draw button uses from then on.

One press is about **24c**, because it is four images. It deliberately does not use the style reference — matching a reference is a different question from which model draws better, and including it would judge every model after the first on how well it copied. If your key cannot use one of the models, that square says so and the other three still arrive.

If you have seen nicer images elsewhere — Google Slides, say — this is usually the whole explanation: those tools are not running a budget model, and the default here is the cheapest of the four.

Keep every scene in the same style. On Google only, and on by default. The first image you draw becomes the reference for every one after it, so a video looks like one set instead of twelve unrelated pictures. It costs nothing extra.

What it costs. On ElevenLabs, every press spends credits from the same balance as your voiceover, and drawing through the API needs a **Pro plan or above**. On the free or Starter plan the voice still works perfectly — only the Draw button will tell you the plan is not enough. There is deliberately no "draw every scene" button: it would be one click to spend a lot of credit on scenes you were going to skip anyway. The one batch button is for the scenes below, where there is nothing else to choose from.

Scenes with no honest photo

When Gemini writes the script it leaves a scene's search words **empty** if no real photograph fits the idea — "*why the current lags the voltage*" has no stock photo, and a misleading one is worse than none. A drawing is not

bound by what a camera can capture, so those scenes are drawn instead.

A panel above the scenes says how many there are, e.g. "✨ *5 scenes have no honest photo — draw them instead*":

1. ✨ **Write drawing prompts for N scenes.** Gemini writes a prompt for each from what that scene says, with the whole script as context — an apparatus, a process made visible, a clear metaphor. This is a text request, so it works on the free tier.
2. **Read them.** Each scene shows its **Drawing prompt**, with **edit** and **rewrite with Gemini**. A single scene can also get its own ✨ **Write a drawing prompt**.
3. 🎨 **Draw and attach N images.** Draws each scene that has a prompt and no picture yet, in the Look and model you picked, and **attaches each to its scene** — the one place anything is applied for you, and only because the button says so. The first picture becomes the style reference for the rest. A progress bar counts them off.

Two rules are enforced whatever Gemini writes: a picture for a scene **before the answer** is never allowed to show the answer (it is replaced with a plainer one, and a note names the scene), and words that make image models paint lettering — *text*, *labels*, *numbers* — are taken out.

Being honest about it. Drawn backdrops are labelled *Generated with AI* in your caption and in the publish kit's credits file. Leave that in. YouTube and the other platforms increasingly expect AI-made imagery to be declared, and the line is short enough that it costs you nothing.

The Doodle look and the mascot

Doodle is a look of its own, and **everything in it is drawn by hand** — not just a mascot pasted onto an ordinary video. A notebook page, words in a handwritten marker font, and pictures that are either the channel's stick-figure engineer or an illustration of the thing being explained. Light is paper; dark is a chalkboard, with the drawings turned into chalk lines.

- **Diagrams, charts and icons** — the worked-out circuits, phasors and bar models, the explainer's step cards and arrows — are inked with a marker wobble, so a wire wavers like a line drawn by hand. The maths under them is untouched: they are still computed, only drawn differently. Their lines gently **boil** — re-traced a few times a second, like hand-drawn animation. Turn *Motion* down to 0 and they hold still.
- **Formulas** are written out in the same marker, not typed in a code font.
- **The answer is marked like a paper**: the right option circled in green marker, then the others struck through one by one.
- **Action words are drawn on** — steam over *heats*, a wavy arrow under *flows*, a bolt beside *sparks* — as each is spoken. See Animation comes from your verbs.
- **The handwriting is set a size larger** than the other looks. Lowercase marker reads smaller than a typed font at the same size, so the words, options and captions are all scaled up to match.

It is built around **one character who never changes**. A recurring face is what makes a run of videos recognisable in a feed, and image models drift — by the sixth picture the hat is gone and the head is a different shape. So the engineer is drawn against a fixed **model sheet** every time.

1. The mascot, once. Press **Mascot** in the top bar. *Draw 4 designs* draws four directions for the character on the best model (about 52c); click the one you want and it becomes the model sheet. It is saved in `public/mascot/` and **committed with the code**, so your other computer draws the same engineer. *Draw 6 test scenes* (about 24c) acts out one question — puzzled, alarmed, thinking, zapped, got it, teaching — so you can see the character hold before you use it for real. You only do this again if you want a different mascot.

2. Choose Doodle on the Look step. The *Backdrop photos* section becomes **Doodles**:

- **Energy** — *Calm*, *Lively* or *Chaotic*: how much sweat, squiggle and zap the drawings get. It is a maximum, and **teaching scenes stay calm whatever it says** — a sparking wire beside the explanation tells the viewer something is dangerous when nothing is. Only the hook and the reveal can go fully chaotic. Each scene shows the energy it will actually be drawn at.
- **Direct N scenes** — Gemini chooses, for each scene, **the engineer or an illustration**, then writes it: for the engineer, what they are *doing*, *feeling*, what they are *with* and the *gag*; for an illustration, what it *shows* — a transformer on a pole, electrons drifting through a wire — with nobody in it. The engineer suits the hook, the question and the reveal; illustrations suit the explaining. Text only, one request. Switch any scene's subject, or edit any field, afterwards.
- **Draw N scenes** — draws every directed scene against the model sheet, three at a time, each appearing as it lands. The price is on the button before you press it. *Stop after these* stops it part way.
- **Redraw** on any one scene draws just that scene again, for one picture's price. No rule stops an image model misbehaving every time, so the fix for one bad drawing is one more drawing.

The animated engineer (the *Animated engineer* switch, on by default). The engineer is no longer drawn into each picture: he stands beside it and **moves**. He is the model sheet rebuilt from lines, so he is exactly the same engineer in every video, and he costs nothing.

- **Directing is enough for him to appear.** Each engineer scene gets a **pose** — *Waving, Pointing, Has an idea, Thinking, Shocked, Cheering, Explaining, Worried, Shrugging* or *Standing* — chosen by Gemini and changeable from the drop-down on the scene. He appears in the preview the moment directing is done; nothing has to be drawn.
- **Only what stands beside him is drawn.** The *Beside him* field names the things on the page next to him — a meter, a lamp, a tired battery — and *Draw* draws just those, with no engineer in the picture. A scene that is only his reaction has nothing beside him, says *Nothing to draw*, and costs nothing. A typical short now needs two or three pictures instead of eight.
- **He is one character through the whole video.** He does not fade out and back in at every cut: he changes pose, slides aside when a scene has something beside him, and steps back to the middle when it does not. He leaves only for scenes that are an illustration, or show their own diagram.
- **He blinks and acts out the action words** as they are said — a hand swept across for *flows*, a finger circling for *spins*, fanning his face for *heats*, a hug for *cools*, a clap for *collides*, arms flung open for *bursts*. He does not talk: a moving mouth on a stick figure looked awkward, so his face holds the scene's expression. Everyday electrical words (*voltage*, *current*, *circuit*) get a mark on the word but do not make him jump — only *sparks* and *lightning* do.
- **He points at what is being explained.** On an explainer scene with a layout — steps, a diagram, a comparison — he points at each part as the narration reaches it, with the arm on that side, and looks where he points.
- **Every action word has a sound**, quiet and under the voice: a whoosh for *flows*, a zap for *sparks*, a hiss for *heats*, a whirr for *spins*, a drip, a thud, a shimmer. The formula cards pop as they land. Made by the studio itself, so nothing is downloaded or licensed; *Sound effects* off on the Look step silences them with the rest.
- **Scenes drawn before he was animated** have him in the picture, so they are shown as that still, never with a second engineer beside them, and are tagged *has him in it*. *Draw* redraws them as the things beside him.
- **Turn the switch off** for the old way: the engineer drawn into every picture, as a still.

To watch him on his own, `npm run engineer:preview -- light --video` renders his test reel — every pose, then every mime — into `stills/`.

The thumbnail and the carousel match. Choose Doodle and both come out on the same notebook page, in the same marker, inked by hand like the video:

- **The thumbnail** puts the engineer *beside* the title — right of a YouTube cover, below a Short's — drawn on the page, not painted behind the words. *Draw the engineer for the cover* (on by default, one picture) draws him with a big reaction to whatever Gemini's design says the picture shows — a face is what stops a thumb in a feed. Turn it off and the cover uses the drawing already made for the video's reveal or hook, for nothing.
- **The carousel** circles the right answer and strikes out the rest on its answer slide, and the last slide ends with the engineer waving under *Follow for one every day* — his own model sheet, so it costs nothing. If the slide has a long fun fact, the fact gets the room and the engineer is left out: the words matter more than the drawing.

What it does on screen. Each scene is split rather than layered: the words get their own band — the top of a portrait frame, the left of a landscape one — and the drawing gets the rest, so a caption can never land on the engineer's face. The drawing is blended *into* the page, not pasted on: its white paper disappears and the notebook dots run through it.

What it leaves out, on purpose. Backdrop photos, the moving backdrop and the drifting symbols are all switched off in the Doodle look — they are ways of filling a frame that has no picture, and every doodle scene has one. A scene that shows **its own diagram** — a worked-out circuit, a chart, a moving sketch — keeps the diagram and gets no doodle: the checked picture wins. The panel lists those scenes.

Rules the drawings follow. Black ink and **one red, which means electricity** — sparks, live wires, the bolt on the hat. **No words in the picture**, and no written sound effects (*zzz*, *zap*, *boom*): image models misspell them, and the words are the renderer's job. Scenes before the answer never show it — a direction that gives it away is dropped, and a note names the scene.

The font. Kalam, stored in `public/fonts/`, so a render never waits on the internet. It covers English only — Kannada and other Indian scripts use the regular font.

To see the look without spending anything, `npm run doodle:preview` renders stills of every kind of scene into `stills/` from the mascot test drawings. `npm run doodle:preview -- portrait light actions action` renders one still for every kind of action mark, each taken just after its word is spoken. `npm run doodle:preview -- portrait light engineer x --video` renders a short quiz with the animated engineer, as stills and as a clip, to see him move through the cuts.

Cuts, text and the finishing layer

Three settings on step 5 that apply to every scene. Because they run on all of them, a choice here is *felt* across a whole video rather than noticed once — which is exactly why the defaults are the quiet options.

The transition — how one scene becomes the next

Ten choices. **Auto** is the default and is usually the right answer: it varies the join by what the scene is doing, so the answer reveal gets the punchy zoom, the question gets a wipe, and the explanations get the quietest crossfade there is. A single transition used forty times becomes a tic by the fourth scene.

	What it does	When
Auto	Varies with the scene	Leave it here unless you want one look throughout
Crossfade	Dissolve with a small drift	Never wrong, never noticed
Slide	The new scene moves in over the old	Clean and modern; reads well on a phone
Push	The old scene is shoved out by the new	More physical; good for step-by-step

	What it does	When
Wipe	A hard edge sweeps across	Graphic and confident; suits bold layouts
Zoom	Punches in through the cut	Energetic; best on short, fast videos
Blur	Defocus and refocus	Soft and expensive-looking; slows the pace
Dip	Through the background colour	A real beat; use when two ideas are separate
Whip pan	Fast sideways smear	High energy — not for twenty scenes
Glitch	Digital break-up	Loud. One or two a video, not every cut

How the words appear

The read-along model never changes: one line on screen, exactly what you are hearing, the spoken word lit. What you are choosing is the *manner of arrival* — and the real decision inside it is whether the viewer may read slightly ahead of the voice.

- **Fade** *(default)* — the phrase appears and words light up as they are said. Unspoken words wait faintly, so the eye can run a shade ahead. Calmest, and the easiest to follow.
- **Pop** — each word snaps in as it is spoken. Punchy; suits hooks.
- **Rise** — words lift into place. Smooth all-rounder.
- **Typewriter** — nothing exists until it is said. Nothing to read ahead to, so it holds attention harder. Use it when you want urgency, not comprehension.
- **Focus** — unspoken words are blurred and sharpen as they are reached.
- **Highlighter** — a bar sweeps behind the current word. The strongest choice for **method and revision videos**: it reads as teaching rather than as motion graphics.
- **Bounce** — springy overshoot per word. Best with the Flashy layout.

The finishing layer

A texture over the top of everything. Unlike the moving backdrop, it means nothing and is not chosen from your subject — it is there to make a frame look *shot* rather than assembled.

Vignette (darkened edges), **film grain**, **scanlines** (CRT), **light leak** (a drifting warm bloom), **cinematic bars** (letterbox), **corner frame** (viewfinder brackets), **dust** (floating specks), **chromatic edges** (a lens fringe). Plus **None**, which is the default and what every video looked like before this existed.

Each has its own ceiling, so **How strong** at maximum cannot make the captions hard to read — a vignette can afford far more than grain can. If you are unsure, vignette at half strength flatters almost everything and is impossible to notice.

> One at a time. These stack with the moving backdrop, the drift symbols and the backdrop photo, and a frame carrying all four is a frame with nothing to look at.

Looks from Claude Design

A design system you make at claude.ai/design can become a layout here. Its colours, fonts, corner radius and alignment come across exactly as published; motion, timing and narration stay with Shorts Studio.

What carries over. The ground, card, text and accent colours are the system's own tokens. The dim text, borders and alternate surfaces come off its neutral ramp, so they have the same visual weight they have in Claude Design. Its layout style sets where text sits: Organic says *flush left*, so Organic scenes are flush left unless you change **Where the text sits**.

The mode it was not designed for is worked out, not guessed. Organic was designed on a light ground. Its dark mode uses the system's own ink as the ground and moves the accent to the step Claude Design recommends for dark grounds. Every look is checked in both modes for readable contrast before it is allowed in.

Right and wrong stay green and red. A system's second accent is used for the correct answer only when it is actually green. A brand with a red accent never marks the right answer in red.

Fonts. The system's webfonts (Caprasimo and Figtree, for Organic) are named first, but they are not installed yet. Until they are, the text falls back to the closest installed face, so a render looks plain rather than broken.

Adding another look

Create or open the design system at claude.ai/design, then ask Claude Code to *pull the design system called …*. It saves the system into `design-kits/<name>/` and runs:

```
node tools/import-design.mjs
```

That rebuilds `src/lib/design-looks.ts`, and the new layout appears on step 5. A kit cannot take the name of a built-in layout, and one that fails the contrast checks stops the tests rather than quietly shipping unreadable text.

> The first time this is done on a computer, Claude Code needs design access once: run `claude` in a terminal and type `/design-login`.

Aptitude and reasoning videos

Almost every competitive exam in India carries an aptitude paper alongside the technical one, and for a lot of candidates that is the paper which actually decides the result — everyone revises their own subject, and nobody practises ratios. **Aptitude & reasoning** on step 2 writes for those sections.

What it covers. Twenty-eight sections, in the three blocks a paper is printed in:

- **Numerical** — number system, simplification, percentage and ratio, averages and alligation, profit and loss, interest, time-speed-distance, time and work, algebra, geometry and mensuration, trigonometry, permutation and probability, data interpretation, data sufficiency.
- **Reasoning** — series, coding-decoding, blood relations and directions, syllogism, puzzles and seating arrangement, analogy and classification, non-verbal, analytical and critical reasoning, clocks-calendars-cubes.
- **The rest of the paper** — English and comprehension, current affairs, static GK, general science, computer awareness.

Each one comes with its own sub-topic list, so you can run a whole series on boats and streams, or on syllogism possibility cases, without typing a topic each time.

Pick the paper, not just the section. The **Exam** dropdown matters more here than it looks. An SSC quant question and a CAT quant question can sit in the same chapter and be nothing alike: SSC wants one clean step in about forty-five seconds with numbers you can hold in your head, while CAT wants a question where the obvious approach is the slow one and there is an insight that collapses the work. Banking rewards approximation over calculation. Placement papers stick to one concept per question. Choosing the wrong one gives you a technically

correct question aimed at nobody.

What the tool is told to do differently. Aptitude questions are marked against a clock, so the question has to be solvable in the time that paper allows — and the wrong options are treated as part of the lesson, not padding. Each one has to be a mistake candidates genuinely make: the ratio inverted, the percentage taken on the wrong base, the units left unconverted, the off-by-one in a series, the answer to the question that was not asked. A distractor nobody would pick makes the question easier than the real paper and teaches nothing.

The explanation is told to name the method out loud — "this is an alligation", "use the LCM method" — because the pattern is the takeaway, not this one answer. Where a shortcut exists it shows the textbook route first, then the shortcut, and says how much time it saves.

Long-form method videos

This is where aptitude pays off most. Switch **How it is told** to **Explainer** and the storyboard is built as a method lesson rather than a curiosity piece:

1. Open on a question of this type and make it clear why the obvious approach is too slow.
2. Name the method in one sentence.
3. Work a full example, one step per scene, on a **steps** panel so the working builds up on screen.
4. Put the shortcut against the long way on a **versus** panel, with the seconds saved.
5. Show the trap this chapter is famous for on a **grid** panel.
6. Work a second, slightly different example so the viewer sees the pattern transfer.
7. Recap the method as numbered steps they can screenshot.

An ordinary explainer is told to use pictures and analogies instead of equations. That instruction is deliberately reversed here: in a method video the arithmetic on screen *is* the picture, and a viewer who cannot see the working cannot copy it. Keep the numbers clean enough to follow without pausing — if the working needs a calculator, the question needs different numbers.

A note on units

Voice models read **10 MW** as "ten mili wag". Any unit symbol with a number in front of it — MW, kV, kWh, MVA, Hz, Ω, °C, % and the rest — is expanded for the **voice only**. The narrator says "ten megawatts"; the screen still shows `10 MW`, and the highlight lands on it at the right moment.

A symbol with no number before it is left alone, so "A transformer" stays an article and does not become "ampere transformer". Write units the way you normally would.

---

5. Git — your undo button

Git takes **snapshots** of your project. It is the reason a mistake can never cost you more than a few minutes.

Three ideas, and that is genuinely all you need:

- **A commit** is a labelled snapshot. "Here is what everything looked like at 4pm."
- **Your history** is the list of those snapshots.
- **You can always go back** to any of them.

Git deliberately ignores three things: `node_modules` (hundreds of megabytes, rebuilt by `npm install`), your finished videos, and generated audio. Everything that matters is tracked.

The only commands you need

See what you have changed:

```
git status
```

Stage everything you changed:

```
git add -A
```

Save the snapshot:

```
git commit -m "describe what you changed"
```

The message is for future-you. "Made the countdown longer" beats "update".

See your history:

```
git log --oneline
```

Undoing things

Throw away changes to one file you have messed up:

```
git restore path/to/file
```

Throw away **all** uncommitted changes and go back to your last snapshot:

```
git restore .
```

> ⚠ That last one is not itself undoable. It discards everything you have changed since your last commit. Commit often and it is never frightening.

---

6. GitHub — your backup and your bridge

GitHub is a website that stores a copy of your project in the cloud. It does two jobs: it is a backup if your laptop dies, and it is how the same project reaches a second computer.

Your project lives at **github.com/HHHkumar/shorts-studio**, and it is **private** — only you can see it.

Send your latest snapshots up:

```
git push
```

Bring down changes made on another machine:

```
git pull
```

The rhythm that keeps it painless

Pull before you start. Push before you stop.

A full session looks like this:

```
git pull
```

...do your work, then...

```
git add -A
```

```
git commit -m "what you changed"
```

```
git push
```

If you forget and edit the same file on both machines, Git will ask you to reconcile the two versions. Irritating, but nothing is ever lost.

> **If a push fails with "Permission denied to <some other name>"**, Windows has a different GitHub account saved. That is exactly why your project's address includes your username: `https://HHHkumar@github.com/HHHkumar/shorts-studio.git`. It tells Git which account to use.

---

7. Running it on another computer

Everything you need is on GitHub, so this is four commands.

1. Open PowerShell where you want the project to live, then:

```
git clone https://HHHkumar@github.com/HHHkumar/shorts-studio.git
```

2. Go into it:

```
cd shorts-studio
```

3. Install the packages (this is why `node_modules` is not in the repo — it is rebuilt here):

```
npm install
```

4. Start it:

```
npm start
```

Four things that do not travel, by design

- **Your API keys.** They live in your browser, never in a file. Paste them into step 1 again.
- **The rendering browser.** The first render on the new machine downloads it again (~150 MB, once).
- **Your finished videos.** They are too big for a repo. They stay on the machine that made them.
- **The Python packages for the guide PDF.** `npm install` fetches everything JavaScript, but the one Python script in the project — the one that rebuilds this document as a PDF — has its own two dependencies. You only need them if you edit the guide:

```
pip install -r tools/requirements.txt
```

After that, both computers are equal. Pull before you start, push before you stop, and they stay in step.

---

8. What it costs

Service	Free allowance	One video uses
Gemini	A generous free tier	One or two requests. Realistically free.
ElevenLabs	10,000 characters a month	A short uses 500–900 characters (**10–20 a month free**). A five-minute explainer uses about 4,400 (**two a month**).
Claude *(optional)*	Pay as you go, **no free tier**	Opus: about **9c** a quiz, **35c** a five-minute storyboard. Sonnet: about **4c** and **14c**.
DeepSeek *(optional)*	Pay as you go, no free tier	A fraction of a cent per check.
Pexels / NASA *(optional)*	Free	Nothing.
Drawn pictures *(optional)* — scene backdrops and thumbnail art	**None.** Needs **billing on the Gemini key**, or an **ElevenLabs Pro plan**	One press, one picture: about **3c–13c** on Gemini depending on the model, or ElevenLabs credits from the voice balance. *Draw and attach* on a video with five no-photo scenes is five pictures.
Doodle scenes *(optional)*	**None.** Needs **billing on the Gemini key**	One picture per scene: about **4c** on Flash. A short of 8–10 scenes is **30–40c**; a 90-second explainer of 15 scenes about **60c**. With the **animated engineer**, his scenes are free unless something is drawn beside him, so a short is more like **8–12c**. Directing them is text — free tier. The mascot itself is a one-off: about 52c to design, 24c to test.
Gemini text extras — metadata for three platforms, thumbnail design, drawing prompts, doodle directions	The same free tier	One request each.
Rendering — videos, thumbnails, carousel slides	Unlimited	Your own computer. Costs electricity.

Changing the look, re-rendering, editing text, picking photos, making thumbnails without a painted picture and making carousels all cost **nothing**, and neither does **Write it myself** — that path needs no Gemini key at all. Only *Generate the question*, *Make the voiceover*, *Check the answer*, *Write the metadata* and anything that **draws a picture** spend anything.

To stretch ElevenLabs credits: shorter targets, and the **Flash** voice model.

9. When something goes wrong

What you see	What it means	What to do
"*running scripts is disabled on this system*"	Windows blocks npm's launcher by default.	Use `npm.cmd`, or run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.
Red box: "*The helper server is not running*"	The helper really is not there — the page waited twenty seconds first.	Reopen PowerShell, `cd` to the folder, `npm start`, reload the page.
"*Port 3030 is already taken*"	A second copy of the tool is still running.	Close the other PowerShell window. The message prints the command to force it if you cannot find it.
"*Port 5173 is already in use*"	Same thing, for the web half.	Close the other window and start again — but see the next row if that does not help.
The port is still taken after you closed the window	Closing a window does not always kill what it started. `npm start` runs **two** child processes, and they can outlive the window and keep listening.	Find and stop them by number — see When a port stays stuck below.
Both halves exit at once, `[APP]` then `[SERVER]`	Normal, and not two faults. The two halves run under `concurrently -k`, so when one fails it deliberately stops the other.	Read the **first** error only. The second exit is the consequence, not the cause.
"*That Claude API key was rejected*"	Bad key, usually a stray space.	Re-copy it from console.anthropic.com.
"*Your Anthropic account has no credit left*"	Claude has no free tier.	Top up, or switch **Who writes it** back to Gemini on step 2.
"*Claude declined this topic*"	A safety classifier refused it.	Reword the topic, or use Gemini for that one.
The **Who writes it** choice is missing	No Claude key is set.	Paste one on step 1; the choice appears by itself.
"*Gemini stopped part way through the script*"	It wandered rather than ran short — the room it is given is now four times what the longest video needs.	Try again. If it repeats on one topic, shorten the target length or change model.
"*This model spent its whole budget thinking*"	A newer Flash model reasoned until it had no room left to write.	Pick a **2.5** model in the dropdown. They are the reliable choice for long scripts.
"*No reply after 240 seconds*"	The model stalled.	Try again, or switch to a Flash model.
"*That … API key was rejected*"	Bad key, usually a stray space.	Re-copy it from the provider and paste again.

What you see	What it means	What to do
"*Your ElevenLabs character quota is used up*"	Out of voice credits this month.	Wait for the reset, shorten the video, or upgrade.
"*ElevenLabs image generation needs a Pro plan or above*"	Drawing backdrops is a paid-plan feature on the API. Your voiceover is unaffected.	Use **Find backdrop photos** instead, or upgrade the plan.
"*ElevenLabs refused that prompt as unsafe*"	A moderation filter rejected the words for that scene.	Reword that scene's search box and press **Draw** again.
"*ElevenLabs was still drawing after 120 seconds*"	The model queued rather than failed.	Press **Draw** again, or pick a **Flash** image model — they answer in seconds.
The **Draw** button is missing on step 5	No key for the service picked in the tiles above — **Google** uses the Gemini key, **ElevenLabs** its own.	Paste that key on step 1, or switch tiles; the button and its dropdowns appear by themselves.
`public\generated\ai\` or `\thumbs\` keeps growing	Drawn images are never deleted automatically, because they cost money.	Delete the folder yourself when you have finished with those videos.
"*Your DeepSeek account has no credit left*"	DeepSeek has no free tier.	Top up, or clear the key to turn the check off.
"*Gemini rate limit hit*"	You generated too fast.	Wait a minute and try again.
"*Your key cannot use that Gemini model*"	The model is not available to your key.	Reload the page — the dropdown rebuilds from your own key.
No voices in the dropdown	Your ElevenLabs account has no voices saved.	Add one from the Voice Library, reload.
Render stuck at 0% the first time	It is downloading the rendering browser.	Wait. It happens once per computer.
"*The output file is locked*"	A video player still has the last render open.	Close it and render again.
A spoken line sounds clipped	Trailing silence trimmed too aggressively.	Step 5 → turn off **Trim trailing silence**.
A diagram shows wrong numbers	Gemini invented them.	Fix that scene in step 3, or turn off **Draw the diagrams**.
A video feels too busy	Too much is moving for the subject.	Step 5 → **Narration effects** → **How strong**. Turn it down; 0 holds everything still.
It ignored something you asked for	Your instruction was outranked, or never landed.	Step 2 → **See exactly what will be sent**. If a built-in line contradicts you, say so more plainly.
A scene feels flat and still	The narration names no movement.	Step 3 → check the ✨ **Moves on these words** row. Use the real verb: *flows*, not *goes*.
An effect fires where it makes no sense	A word matched that you did not mean physically.	Reword that phrase; the row shows which word did it.

What you see	What it means	What to do
A moving scene is out of step with the voice	A cue is missing from that scene's narration.	The window says which words. Put them back, or regenerate.
A box has no picture in it	No icon matched that noun, or the storyboard named an abstract idea.	Normal for abstractions. Otherwise use a plainer noun.
A moving scene has a blank circle in it	No icon matched that word.	Step 3 → reword it to a plainer noun, e.g. "fish" not "salmonid".
No moving scenes ever appear	The subject may not have a moment that moves — or the model skipped it.	Step 2 → **Your own instructions** → *act out the mechanism with a moving scene*.
The script is far shorter than asked	Gemini underwrote it.	Step 3 warns you. Regenerate, or switch to a stronger model.
Push fails: "*Permission denied to …*"	Windows has another GitHub account saved.	Make sure the address includes `HHHkumar@`.
The kit has no chapters	The video is too short, or YouTube's rules cannot be met.	Normal on Shorts. Chapters need 60s+, three marks, ten seconds each.
Pack the upload kit is greyed out	The metadata has not been written yet.	Press **Write the metadata** first — the kit is built from it.
The kit has no thumbnail in it	It was packed before you made one.	Make the thumbnail, then press **Pack it again**.
The kit has no `carousel\` folder, or no `instagram.txt`	The carousel was made, or the metadata written, after packing — or the metadata predates the Instagram tab.	Make the carousel / press **Write it again**, then **Pack it again**.
Step 3: ✗ "*The diagram does not agree with the answer*"	The worked-out diagram and the marked option differ. One of them is wrong.	Check the question before recording. Fix the answer, or generate again. The video simply has no diagram until they agree.
Step 3: ⚠ "*The diagram was not drawn*"	The model described a setup that cannot be drawn honestly.	Generate again. The video is fine without it.
A scene that should have an animation has none	Its labels made it meaningless and it was dropped — most often a two-signal animation (waveform, phasor) labelled with two measures of **one** wave, such as *Peak* against *RMS*, which cannot be out of phase with each other.	Nothing to do: the scene plays with its words alone. If you want a picture there, step 5 → draw one for that scene.
Step 7, Instagram tab: "*No Instagram text yet*"	The metadata was written before that tab existed.	Press **Write it again**.
"*Google would not draw that image*" on **Design with Gemini**, **Repaint** or **Draw and attach**	Image generation is not on the Gemini free tier.	Enable billing on the key's project — or untick **Paint a background picture** for a free design without one.
Draw and attach stopped part-way	One picture failed (usually billing or a rate limit); the ones before it are attached.	Read the red note, fix it, press **Draw and attach** again — it only draws the scenes still without a picture.

What you see	What it means	What to do
Keys and **Topic** are greyed out at the top	Those two only unlock with both a Gemini and an ElevenLabs key present — so after **Write it myself** without keys there is no chip back to step 1.	Reload the page: it always opens on step 1, and your script is kept.
A new button or step does nothing, or says "*Request failed (404)*"	The helper was started before the tool was updated. The page updates by itself; the helper does not.	`Ctrl + C` in PowerShell, then `npm start` again.
A change seems to have no effect	An old server is still running from before.	`Ctrl + C` in PowerShell, then `npm start` again.
Everything is confusing	—	Step 7 → **Reset everything**, then start from step 1.

When a port stays stuck

`npm start` is not one program. It starts **two** — the helper server on 3030 and the web app on 5173 — and closing the window they were launched from does not always take them with it. They keep running, invisibly, still holding both ports. The next `npm start` then fails on 5173 before it has done anything, and because the two halves are deliberately tied together, the helper is stopped too. That used to show up as a Vite stack trace and two red exits for one problem.

`npm start` **now checks first.** Before it launches anything it looks at both ports, and if one is taken it stops and says so plainly — which port, and whether it is **an older Shorts Studio that is still running** or **another program** — with the command that fixes it. For the usual case, a leftover studio:

```
npm run free-ports
npm start
```

`free-ports` stops **only Shorts Studio's own** leftovers — the web app and the helper, recognised by their command lines — and names each one it stops. If another program is on the port it is left alone and named instead, because stopping it is your call, not the tool's.

To see or stop things by hand — for that other program, or if you are curious — find what is holding the ports:

```
netstat -ano | findstr "5173 3030"
```

The last number on each line is the process ID. Stop each one:

```
taskkill /PID 21044 /F
```

Then `npm start` as normal. Nothing is lost by doing this — those processes hold no unsaved work. Your videos are in `out\`, your keys are in the browser, and everything else is on disk already.

> If this happens every time you close the window, get into the habit of stopping the tool with `Ctrl + C` in its own window rather than clicking the X. `Ctrl + C` shuts both halves down properly; the X sometimes only closes the window around them.

Checking the build itself

If something feels broken and you want to know whether it is the tool or the model, run:

```
npm test
```

The fastest one. It checks the rules — the cleaners, the timings, the chapter arithmetic, the animation pacing — without drawing anything. A few seconds, no API key, no network.

```
npm run simulate
```

Feeds a complete storyboard — deliberately including a few of the mistakes a model really makes — through every stage the real thing uses: the cleaner, the motion check, the icon fetch, the timeline, the upload kit and the renderer. Add `-- --render` to also encode a real video from it, which takes a few minutes and proves the whole chain end to end.

```
npm run smoke
```

The widest one, and the one to run after any change to how videos are put together. It renders **both kinds of video in both shapes** — quiz and explainer, 9:16 and 16:9 — using an explainer that deliberately contains every layout, plus all eight thumbnails. It then checks that no frame came out blank, which is the failure that hides: a scene that renders successfully and draws nothing looks exactly like a scene that is fine, until you watch it.

It leaves the frames in `stills\smoke\` so you can look at them yourself. That is worth doing — the checks catch a blank frame, they cannot tell you a layout is ugly.

Every line of all three should say `ok`.

None of them can call Gemini, Claude, ElevenLabs or DeepSeek — the keys live in your browser, not on disk — so they prove everything *around* the models, not the models' answers. That is what step 3 is for.

> **Last full test: 17 September 2026.** `npm test` — 1,360 checks, about 15 seconds. `npm run smoke` — every video kind and shape plus all eight thumbnails, none blank. `npm run simulate -- --render` — every stage, ending in a real 73-second video. On top of those: all nine diagram kinds drawn at the question and the reveal with every worked-out answer matching its option, five kinds of carousel, thumbnails over drawn art in every layout, every helper route that needs no paid key, and all seven steps of the page. Everything passed.

---

10. Where things are saved

Inside `C:\Projects\shorts-studio`:

- `out\` — your finished videos. **This is the folder you want.** Nothing here is ever deleted automatically, and nothing here goes to GitHub. Beside the videos: thumbnails (`thumbnail-….png`), upload kits (`…-upload-kit.zip`), and one `carousel-…\` folder per carousel, holding its slides and its zip.
- `library\` — the video library: one small file per video you have made. Committed with the code, so your other computer has the same list after a `git pull`.
- `bench\` — the test bench: the questions, the reviewer's verdicts and your marks. Committed with the code. Its frames are in `out\bench\`.

- `out\…job.json` — one note per render, so a render the helper was stopped in the middle of can be run again (see Step 6).
- `public\generated\` — voiceover clips and pictures.
  - `vo-…\` and `stock\` are **cleared automatically after a day** — except the voice of any video in the library, which is kept so it still plays when you reopen it. Both are free to make again.
  - `ai\` (drawn backdrops) and `thumbs\` (painted thumbnail art) are **never cleared**, because those pictures cost money and deleting them on a timer would spend it twice. They grow until you empty them yourself.
- `public\audio\` — the music beds and effects, regenerated on first boot.
- `.cache\` — the icon drawings, kept so the same noun is never fetched twice. Safe to delete; it refills itself.
- `stills\` — frames written by the checks in section 9. Scratch, safe to delete.
- `.git\` — your snapshots. Do not touch it; that is Git's business.
- Your keys and settings — in your browser, not in any file.

---

11. Questions people ask

Can I use my own voice? Yes. Clone it on the ElevenLabs website and it appears in the step 4 dropdown.

Can I change the fonts and colours? Yes. `src\lib\theme.ts` holds every visual choice, with comments. Save it and the preview updates instantly. If you break it, `git restore src/lib/theme.ts` puts it back.

Does it work offline? No. Gemini, ElevenLabs and the one-time renderer download all need the internet. Re-rendering a video you already generated works offline.

Is my data going anywhere? Your topic settings go to Google, your script to ElevenLabs, your question to DeepSeek if you enabled it. When you press **Write the metadata**, **Design with Gemini** or **Write drawing prompts**, the question and script go to Google again to write them; when you press **Draw** or **Repaint**, that picture's description goes to whichever service draws it. Nothing else leaves the computer — the helper server listens only on `127.0.0.1`, so nothing on your network can reach it.

Why is my project not in OneDrive any more? OneDrive was syncing `node_modules` — tens of thousands of files — which is slow and can lock files mid-render. Git does the job properly, so the project moved to `C:\Projects`.

Can I sell the videos I make? Check each service's terms. Note in particular that **Remotion is free for individuals and small teams but needs a paid company licence beyond that** — see https://remotion.dev/license.

Something is still wrong. Look at the PowerShell window. The last few red lines usually say plainly what failed.

---

12. A checklist for good videos

- Curiosity factor at **8 or 9**. Boring questions do not get watched.
- 40–50 seconds for Shorts; 180–240 for explainers.
- **Read the question in step 3 and verify the answer yourself.**
- Run the DeepSeek check — a disagreement between two models is the cheapest bug report you will get.
- Keep the read-along text on; most viewers are on mute.
- Only pick backdrop photos that genuinely fit. An unrelated one makes it look worse, not better.

- If you draw backdrops, keep one **Look** for the whole video. Mixed styles read as carelessness.
- Read the drawing prompts before **Draw and attach** — reading is free, drawing is not.
- If step 3 shows ✕ on the diagram, stop and check the answer. It is the cheapest second opinion you will get.
- Judge the thumbnail in the small preview box, not the big one.
- Post the carousel the same day as the Reel, slides in order, with no more than five hashtags.
- For an aptitude video, check the wrong options are real mistakes. If one is obviously silly, the question is easier than the paper it is meant to prepare you for.
- Thinking time of 3–5 seconds. Longer and people scroll away.
- **Commit and push when you finish.** It takes ten seconds and it is the whole safety net.